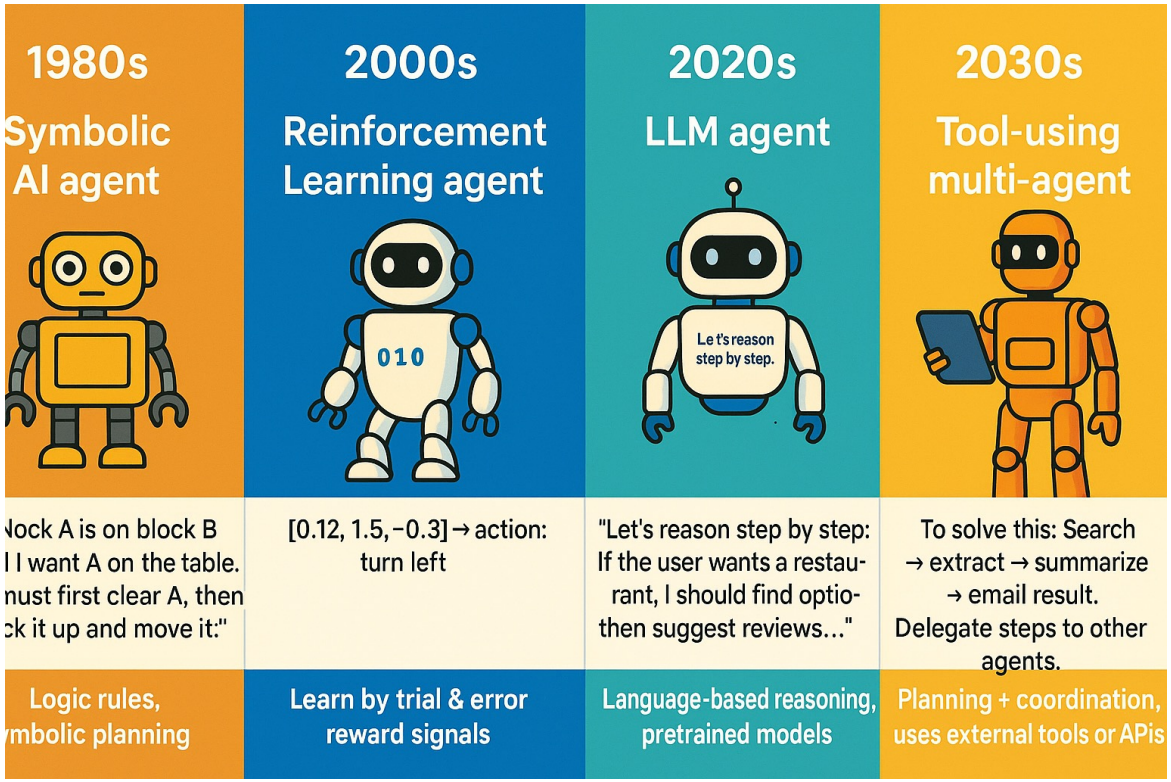
# Classical Planning

CS4246/CS5446

AI Planning and Decision Making

V 4.0

# Types of AI Agents



# Symbolic Agent: Logical Inference

Symbolic AI and Logical Reasoning

Source: https://youtu.be/WHCo4m2VOws?si=aXyXVkKwUrHQzRvX

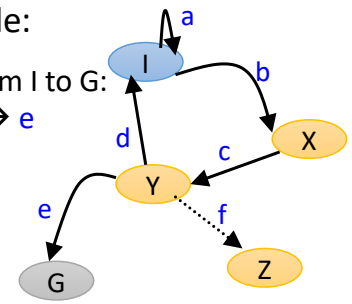
# Topics

| | |
|---|---|
| Classical planning model | Planning model and definition language<br>Factored plan representation |
| Planning as search | Forward state-space search<br>Backward search |
| Planning as logical inference | SATPlan and satisfiability |
| Characteristics of planning | Soundness and completeness |
| From Classical to Modern Planning | Planning frameworks<br>Large foundation model (LFM)-assisted planning |

# Classical Planning

Example:

Plan from I to G:
b → c → e




- Definition:
  - Find a sequence of actions to achieve a goal in an environment that is:
    - Discrete
    - Deterministic
    - Static
    - Fully observable

Planning Domain Definition Language

Stanford Research Institute Problem Solver

- Main challenges:
  - How to represent the planning problem?
  - How to search for a solution (plan)?
  - How to use heuristics to solve for a solution (plan) more efficiently?
  - How to use abstraction (hierarchy) to manage complex plans?

# A Bit of History

- Classical planning – still very useful today!
  - Also called STRIPS planning
- What is STRIPS?
  - STanford Research Institute Problem Solver
  - R. Fikes and N. Nilsson (1971). STRIPS: a new approach to the application of theorem proving to problem solving. Art. Intell, 2:189-208
  - Included planning domain definition language(PDDL) for automated planning problems
- Video:
  - AI Planning – A*Star and STRIPS by Nils Nelson
  - https://media.ed.ac.uk/media/1_uhxvxo4a

# Some Current Challenges

https://deepmind-pushworld.github.io/play/

# Planning Model

STRIPS Planning and PDDL

# Example: Planning Problem Representation

- A classical planning problem

Partial specifications

| | Examples |
|---|---|
| State | $At(P_1, SFO) \wedge At(P_2, SIN)$ |
| Action | $Action(Fly(P_1, SFO, SIN)$<br>PRECOND: $At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport\ (SIN)$<br>EFFECT: $\neg At(P_1, SFO) \wedge At(P_1, SIN)$ |
| Goal | $\neg At(P_1, SFO) \wedge At(P_1, SIN) \wedge At(p, SFO) \wedge Plane(p)$ |

# Planning Domain Definition Language (PDDL)

- **Planning Domain Definition**

```
(define (domain flight_domain)
  (:requirements :strips :typing)

  (:types
    plane
    airport)

  (:predicates
    (At ?p - plane ?a - airport)
    (Plane ?p - plane)
    (Airport ?a - airport))

  (:action Fly
    :parameters (?p - plane ?from - airport ?to - airport)
    :precondition (and (At ?p ?from)
                       (Plane ?p)
                       (Airport ?from) (Airport ?to))
    :effect (and (not (At ?p ?from)) (At ?p ?to))))
```

- **Planning Problem Definition**

```
(define (problem flight_problem)
  (:domain flight_domain)

  (:objects
    P1 P2 - plane
    SFO SIN - airport)

  (:init
    (At P1 SFO)
    (At P2 SIN)
    (Plane P1)
    (Plane P2)
    (Airport SFO)
    (Airport SIN))

  (:goal
    (and (At P1 SIN) (not (At P1 SFP))
         (At ?p SFO) (Plane ?p))))
```

# Main Characteristics

- PDDL:
  - Standard language for planning, derived from STRIPS; many extensions
- Logic-based representation:
  - Compact, lifted form of propositional logic (restricted FOL)
- Factored state representation:
  - World described as state variables or fluents
  - Fluents = properties that change over time
- Implicit transitions:
  - Actions define how states change
  - Other states generated as needed
- Domain-independent heuristics:
  - General methods to guide search across different domains

# State Representation

- State
  - A state is a conjunction of ground atomic fluents or a set of fluents that are true.
  - Ground atomic: a predicate with only constant arguments; function-free.
  - Examples:

| Logical Statement | Set of Fluents |
|---|---|
| $Hungry \wedge Sleepy$ | $S = \{Hungry, Sleepy\}$ |
| $New(Plane1) \wedge Safe(Plane1)$ | $S = \{New(Plane1), Safe(Plane1)\}$ |
| $At(Plane1, SIN) \wedge At(Plane2, SFO)$ | $S = \{At(Plane1, SIN), At(Plane2, SFO)\}$ |

- Database Semantics
  - Closed-world assumption (CWA): Any fluent not in the set is false.
    - Example: $Fierce(CS3263_Lecturer)$ is false.
  - Unique names assumption (UNA): Different constants denote different objects.
    - Example: $Plane1 \neq Plane2$.

# Goal Representation

$\alpha$ entails $\beta$
$\alpha \vDash \beta$ iff in every model in which $\alpha$ is true, $\beta$ is also true
$\alpha \vDash \beta$ iff $M(\alpha) \subseteq M(\beta)$

- Definition:
  - A goal $g$ is a partially specified state, written as a conjunction of literals.
  - A state $s$ satisfies $g$ if $s$ entails (contains all literals in) $g$:

$$s \vDash g$$

- Variables:
  - Variables in goals are treated as existentially quantified.
  - Example: $At(P_1, SIN) \wedge At(p, SFO) \wedge Plane(p)$
- Examples:
  - $Hungry \wedge Sleepy \wedge Bored \;\vDash\; Hungry \wedge Bored$
  - $At(Cargo_1, SFO) \vDash At(c, SFO)$ with *substitution* $\{c/Cargo_1\}$
- Note: For STRIPS, goals are limited to positive, ground literals (no variables).

# Action Schemas and Grounded Actions

- Action schema
  - Defines a family of actions with variables (universally quantified).
- Grounded action
  - Schema instantiated with concrete objects.
- Schema Example:

$Action(Fly(p, from, to)$

Precond: $At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport\ (to)$

Effect: $\neg At(p, from) \wedge At(p, to))$

- Grounded Examples:

$Action(Fly(P1, SFO, SIN)$

Precond: $At(P1, SFO) \wedge Plane(P1) \wedge Airport(SFO) \wedge Airport\ (SIN)$

Effect: $\neg At(P1, SFO) \wedge At(P1, SIN))$

$Action(Fly(P2, SIN, SFO)$

Precond: $At(P2, SIN) \wedge Plane(P2) \wedge Airport(SIN) \wedge Airport\ (SFO)$

Effect: $\neg At(P2, SIN) \wedge At(P2, SFO))$

# Action Effects

$\alpha$ entails $\beta$
$\alpha \vDash \beta$ iff in every model in which $\alpha$ is true, $\beta$ is also true
$\alpha \vDash \beta$ iff $M(\alpha) \subseteq M(\beta)$

- Applicability:
  - Action $a$ is applicable in state $s$ if:

$$s \vDash \text{Precond}(a)$$

  - (i.e., $s$ entails the precondition of $a$ )
- Execution Result
  - Executing action $a$ in state $s$ produces a new state $s'$:

$$s' = (s - DEL(a)) \cup ADD(a)$$

  - DEL(a): fluents made false (negative effects)
  - ADD(a): fluents made true (positive effects)
  - New states are generated on demand by applying add and delete lists of an action.

# Action Execution

- Current state and available action:

  $At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport\ (SIN)$

  $Action(Fly(P_1, SFO, SIN)$

  PRECOND: $At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport\ (SIN)$

  EFFECT: $\neg At(P_1, SFO) \wedge At(P_1, SIN))$

- Apply action

  $Fly(P_1, SFO, SIN)$

- Remove:

  $At(P_1, SFO)$

- Add:

  $At(P_1, SIN)$

- Updated state:

  $At(P_1, SIN) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport\ (SIN)$

$$s' = (s - DEL(a)) \cup ADD(a)$$

# The Frame Problem

- Challenge:
  - Specify what changes after an action without restating everything unchanged.
- In PDDL:
  - Closed-world assumption: unmentioned facts are false
  - Fluents have no explicit time reference
  - Preconditions hold at time $t$; effects apply at time $t + 1$
- Key idea:
  - Only list changes; all other facts are assumed unchanged.

# Example: Air Cargo Transport Planning

$Init(At(C_1, SFO) \wedge At(C_2, SIN) \wedge At(P_1, SFO) \wedge At(P_2, SIN)$
$\quad \wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Plane(P_2)$
$\quad \wedge Airport(SIN) \wedge Airport(SFO))$
$Goal(At(C_1, SIN) \wedge At(C_2, SFO))$

$Action(Load(c, p, a),$
$\quad \text{PRECOND}: At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$
$\quad \text{EFFECT}: \neg At(c, a) \wedge In(c, p))$
$Action(Unload(c, p, a),$
$\quad \text{PRECOND}: In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$
$\quad \text{EFFECT}: At(c, a) \wedge \neg In(c, p))$
$Action(Fly(p, from, to),$
$\quad \text{PRECOND}: At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$
$\quad \text{EFFECT}: \neg At(p, from) \wedge At(p, to))$

Planning Domain

Planning Problem

Adapted from source: RN Figure 11.1

# Example: Planning Domain in PDDL

```
(define (domain air-cargo)
  (:requirements :strips :typing)
  (:types plane airport cargo)
  (:predicates
    (At ?x - (either plane cargo) ?a - airport)
    (In ?c - cargo ?p - plane)
    (Cargo ?c - cargo)
    (Plane ?p - plane)
    (Airport ?a - airport))
 (:action Load
    :parameters (?c - cargo ?p - plane ?a - airport)
    :precondition (and (At ?c ?a) (At ?p ?a) (Cargo ?c) (Plane ?p) (Airport ?a))
    :effect (and (not (At ?c ?a)) (In ?c ?p)))
  (:action Unload
    :parameters (?c - cargo ?p - plane ?a - airport)
    :precondition (and (In ?c ?p) (At ?p ?a) (Cargo ?c) (Plane ?p) (Airport ?a))
    :effect (and (At ?c ?a) (not (In ?c ?p))))
  (:action Fly
    :parameters (?p - plane ?from - airport ?to - airport)
    :precondition (and (At ?p ?from) (Plane ?p) (Airport ?from) (Airport ?to))
    :effect (and (not (At ?p ?from)) (At ?p ?to))))
```

# Example: Planning Problem in PDDL

```
(define (problem air-cargo-problem)
  (:domain air-cargo)

  (:objects
    C1 C2 - cargo
    P1 P2 - plane
    SFO SIN - airport)

  (:init
    (At C1 SFO)
    (At C2 SIN)
    (At P1 SFO)
    (At P2 SIN)
    (Cargo C1)
    (Cargo C2)
    (Plane P1)
    (Plane P2)
    (Airport SFO)
    (Airport SIN))

  (:goal
    (and (At C1 SIN) (At C2 SFO))))
```

# A Possible Plan

- A solution –

- Some caveats
  - What to do with spurious actions like $Fly(P_1, SIN, SIN)$?

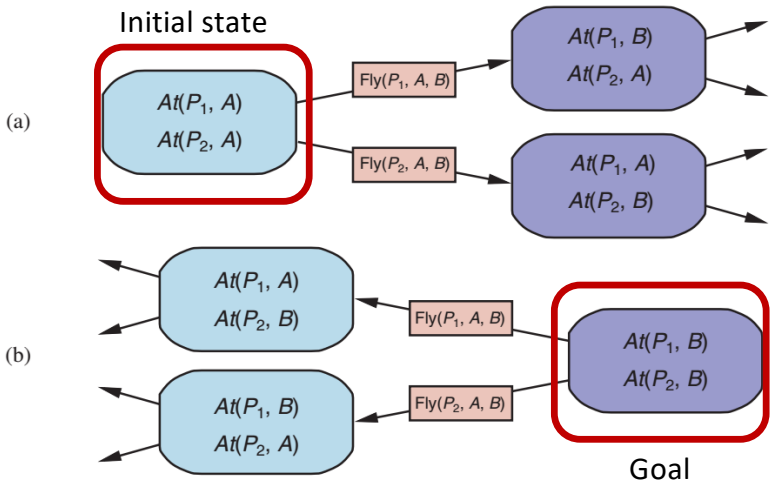
# Planning as State-Space Search

Forward (progression) search

Backward (regression) search

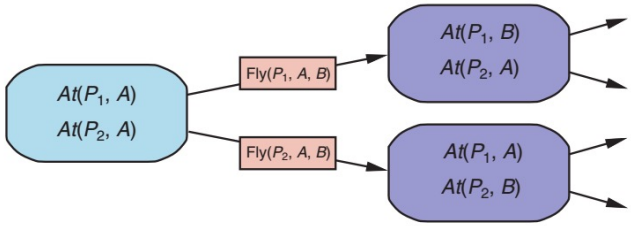
# Planning as State-Space Search

- Planning as Search:
  - Mapping: Planning problem → search problem over states.
  - States: Ground states with binary fluents.
  - Goal: State that includes all positive goal fluents.
  - Actions: Ground actions applicable in each state.
- Search Tree:
  - Nodes = states (initial, goal, intermediates)
  - Edges = actions (state transitions)
  - Plan = path from initial state to goal state
- Algorithms:
  - Forward search (progression)
  - Backward search (regression)




Source: RN Figure 11.5

# Planning by Forward Search




- From Start → Goal using Actions
- Begin at Initial State
  - If $State \models Goal$ (or $Goal \subseteq State$), you're done.
- Find Applicable Actions
  - Action is applicable if $State \models Precond(a)$ (or $Precond(a) \subseteq State$).
- Generate Successor States
  - Apply effects:
  - $s' = (s - DEL(a)) \cup ADD(a)$
- Move Forward
  - Pick a successor as the new current state.
- Key Idea:
  - March forward by adding what actions give and removing what they take away.

# Example

**Forward search:**

Start at $s_0$ → choose applicable action → update with ADD/DEL → reach goal in $s_1$

- Initial State:
  - $s_0 = In(C_2, P_1) \wedge At(P_1, SFO) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Airport(SFO)$
- Goal:
  - $At(C_2, SFO)$
- Relevant Action Schema:

  Action($Unload(c, p, a)$
  Precond: $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$
  Effects: $At(c, a) \wedge \neg In(c, p)$ )
- Grounded Action:

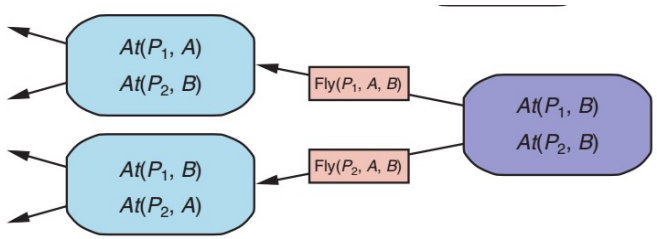
  Action($Unload(C_2, P_1, SFO)$
  Precond: $In(C_2, P_1) \wedge At(P_1, SFO) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Airport(SFO)$
  Effects: $At(C_2, SFO) \wedge \neg In(C_2, P_1)$)

Substitution
$\theta = \{c/C_2, a/SFO\}$

$s' = (s - DEL(a)) \cup ADD(a)$

- New State:
  - $s_1 = At(C_2, SFO) \wedge At(P_1, SFO) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Airport(SFO)$

# Planning by Backward Search




- From Goal ← Start by Regressing Goals
- Begin at Goal Description
  - If $Goal \vDash Initial\ State$ (or $Initial\ State \subseteq Goal$), you're done.
- Find Relevant Actions
  - At least one effect matches the goal.
  - No effect contradicts the goal.
- Regress the Goal
  - Replace achieved effects with preconditions:
  - $POS(g') = \big(POS(g) - ADD(a)\big) \cup POS\big(Precond(a)\big)$
  - $NEG(g') = (NEG(g) - DEL(a)) \cup NEG(Precond(a))$
- Move Backward
  - Pick a predecessor as the new current goal.
- Key Idea:
  - Step back in time—replace parts of the goal with what you need to make it true.

# Example

**Backward search:**
Start at goal g → choose relevant action → regress goal using ADD/DEL and preconditions → reach initial state s0.

- Initial state:
  - $s_0 = In(C_2, P_1) \wedge At(P_1, SFO) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Airport(SFO)$
- Goal
  - $At(C_2, SFO)$
- Relevant action schema
  - $Action(Unload(c, p, a)$
  - Precond: $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$
  - Effect: $At(c, a) \wedge \neg In(c, p)$
- New action schema after substitution
  - $Action(Unload(C_2, p', SFO)$
  - Precond: $In(C_2, p') \wedge At(p', SFO) \wedge Cargo(C_2) \wedge Plane(p') \wedge Airport(SFO)$
  - Effect: $At(C_2, SFO) \wedge \neg In(C_2, p')$
- Regressed state (new goal)
  - $g' = In(C_2, p') \wedge At(p', SFO) \wedge Cargo(C_2) \wedge Plane(p') \wedge Airport(SFO)$

Substitution
$\theta = \{c/C_2, a/SFO\}$

POS(g') = (POS(g) – ADD(a)) U POS(Precond(a))
NEG(g') = (NEG(g) – DEL(a)) U NEG(Precond(a))

# Notes: Relevant States in Regression

- Relevant states:
  - In regression, work with sets of states described by conditions (not single states).
- State vs. Description:
  - State: Every fluent has a truth value.
  - Description: Specifies only some fluents; unmentioned ones can vary.
- Counts:
  - $n$ fluents → $2^n$ ground states.
  - $n$ fluents → $3^n$ descriptions (positive, negative, or unmentioned).
- Example:
  - $[\neg Hungry \wedge Sleepy]$ = all states where Hungry = False and Sleepy = True; other fluents may be any value.

# Notes: Relevant Actions in Regression

- When is an action relevant?
  - Can be the last step before achieving the goal.
  - At least one effect matches (unifies with) a goal literal.
  - No effect contradicts any goal literal.
  - Use the most general unifier (MGU) to reduce branching without losing solutions.
- Regression step: Given goal $g$ and action $a$:
  - Remove literals achieved by $a$.
  - Add $a$'s preconditions.
  - Result is a new subgoal $g'$.

$$Pos(g') = \big(Pos(g) - Add(a)\big) \cup Pos\big(Pre(a)\big)$$
$$Neg(g') = \big(Neg(g) - Del(a)\big) \cup Neg\big(Pre(a)\big)$$

# Notes: Another Regression Example

- Example:
  - Goal: $g = \{At(C_1, SFO)\}$
  - Action: $Unload(c, p, a)$
    Precondition: $In(c, p) \wedge At(p, a)$
    Effects: $At(c, a)$
- Unify:
  - $At(C_1, SFO)$ with $At(c, a) \rightarrow \theta = \{c/C_1, a/SFO\}$
  - Substituted action: $Unload(C_1, P_1, SFO)$
- Regression:
  - Remove $At(C_1, SFO)$ (achieved by action).
  - Add preconditions: $\{In(C_1, P_1), At(P_1, SFO)\}$.
  - $g' = \{In(C_1, P_1), At(P_1, SFO)\}$.

Substitution
$\theta = \{c/C_1, a/SFO\}$

POS(g') = (POS(g) – ADD(a)) U POS(Precond(a))

# Planning as Logical Inference

Boolean satisfiability and SAT-based planners

# Planning as Boolean Satisfiability

If a sentence $\alpha$ is true in model m:
m satisfies $\alpha$
m is a model of $\alpha$
Let $M(\alpha)$ be the set of all models of $\alpha$

- Main Ideas:
  - Convert a planning problem into a propositional formula in Conjunctive Normal Form (CNF).
    - Example CNF: $(P \vee Q) \wedge (\neg Q \vee R \vee S) \wedge (\neg R \vee \neg P)$
  - Check if the formula is satisfiable (truth assignment exists) that corresponds to a valid plan

Main idea:
Reduced planning to the SAT problem

- Note:
  - SAT is NP-complete, but modern solvers handle millions of variables.
  - Uses domain-independent heuristics for efficient search.

# The SATPlan Algorithm

```
function SATPLAN(init, transition, goal, T_max) returns solution or failure
  inputs: init, transition, goal, constitute a description of the problem
          T_max, an upper limit for plan length

  for t = 0 to T_max do
    cnf ← TRANSLATE-TO-SAT(init, transition, goal, t)
    model ← SAT-SOLVER(cnf)
    if model is not null then
      return EXTRACT-SOLUTION(model)
  return failure
```

Assignment of values to variables

Source: RN Fig 7.22

# Example: Eat a Cake!

$Init(\quad)$ ; % $\neg Have(Cake)$ by Closed world assumption

NotHave(Cake)

$Goal(\neg Have(Cake) \wedge Eaten(Cake))$

$Action(Eat(Cake)$

Precond: $Have(Cake)$

Effect: $\neg Have(Cake) \wedge Eaten(Cake))$

$Action(Bake(Cake)$

Precond: $\neg Have(Cake)$

Effect: $Have(Cake))$

# Example: Eat a Cake!

Bounded planning problem with possible solution at n = 2

- Initial and goal states:

| | | |
|---|---|---|
| (Init) | {} | % ¬Have(Cake, 0) |
| (Goal) | ¬Have (Cake, 2) ∧ Eaten(Cake, 2) | |

- The actions:

| | | |
|---|---|---|
| (Eat) | Eat(cake, 0) | ➔ Have(Cake, 0) ∧¬Have(Cake, 1) ∧ Eaten(Cake, 1) |
| | Eat(cake, 1) | ➔ Have(Cake, 1) ∧¬Have(Cake, 2) ∧ Eaten(Cake, 2) |
| (Bake) | Bake(cake, 0) | ➔ ¬Have(Cake, 0) ∧Have(Cake, 1) |
| | Bake(cake, 1) | ➔ ¬Have(Cake, 1) ∧Have(Cake, 2) |

# Example: Eat a Cake! (cont.)

- Successor state axioms : (How fluents can change)

  $\text{Have}(\text{Cake}, 1) \Leftrightarrow (\text{Have}(\text{Cake}, 0) \wedge \neg\text{Eat}(\text{Cake}, 0)) \vee \text{Bake}(\text{Cake}, 0)$

  $\text{Have}(\text{Cake}, 2) \Leftrightarrow (\text{Have}(\text{Cake}, 1) \wedge \neg\text{Eat}(\text{Cake}, 1)) \vee \text{Bake}(\text{Cake}, 1)$

  $\text{Eaten}(\text{Cake}, 1) \Leftrightarrow (\text{Eaten}(\text{Cake}, 0)) \vee \text{Eat}(\text{Cake}, 0)$

  $\text{Eaten}(\text{Cake}, 2) \Leftrightarrow (\text{Eaten}(\text{Cake}, 1)) \vee \text{Eat}(\text{Cake}, 1)$

- Action exclusion axioms: (Assume not eat and bake at same time!)

  $\neg\text{Eat}(\text{Cake}, 0) \vee \neg\text{Bake}(\text{Cake}, 0)$

  $\neg\text{Eat}(\text{Cake}, 1) \vee \neg\text{Bake}(\text{Cake}, 1)$

# SATPlan Summary

- Frame Problem:
  - Most actions do not change most fluents.
  - For $m$ actions and $n$ fluents:
  - Naive: one axiom per fluent unaffected by each action → $O(mn)$ axioms.
  - Successor-State Axioms:
  - A clever encoding that handles the frame problem well.
  - Encode only actions affecting a fluent → $O(n)$ axioms (longer but fewer)
- SATPlan
  - Finds reachable action sequences; returns shortest plan if exists.
  - Works in fully observable or sensorless settings.
  - Limits: may include impossible actions; high memory usage.

# Real-World Applications

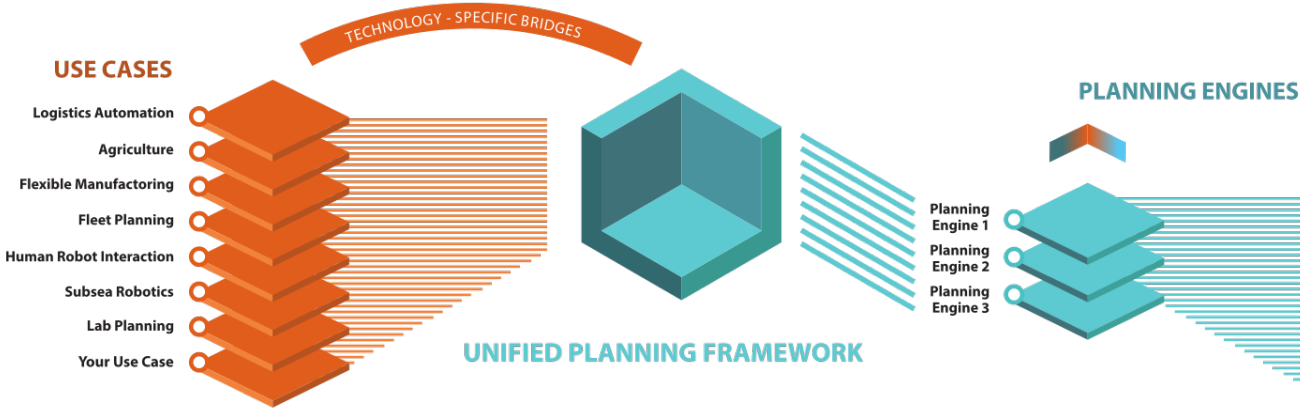
| Domain | Key Use Case |
|---|---|
| Logistics/Manufacturing | Task scheduling. resource management and production planning |
| Enterprise | Business process and work flow planning |
| Robotics | Path planning, autonomous task execution |
| Healthcare | Operating room scheduling, clinical guidelines execution |
| Gaming | NPC behavior and action planning |
| Space Missions | Autonomous mission planning |
| Real-Time Decision Making | Constraints-aware decision support |

# Notes: Real World Applications

- Logistics and Manufacturing: scheduling, resource management, production planning
  - https://www.scitepress.org/Papers/2022/115980/115980.pdf
  - https://www.sciencedirect.com/science/article/abs/pii/S0169023X25000102
- Enterprise: AI Planning for Enterprise [IJCAI 2019]
  - https://www.ijcai.org/proceedings/2019/897
- Robotics: navigation, manipulation [Nature 2025]
  - https://www.nature.com/articles/s41598-025-96614-2
- Healthcare: operating room scheduling, clinical guidelines execution
  - https://arxiv.org/abs/2105.02283
  - https://ceur-ws.org/Vol-3883/paper3_RCRA8.pdf
- Games: NPC behavior planning
  - https://ojs.aaai.org/index.php/AIIDE/article/view/18724
- Space: Mars 2020 scheduler
  - https://ai.jpl.nasa.gov/public/projects/m2020-scheduler/

# AIPlanning4EU Project

Making planning applicable for everyone




Source: https://www.aiplan4eu-project.eu

https://youtu.be/nhO_F9SHEdM?si=-k3_1_bb7P6ZbNmx

## The AIPlan4EU Unified Planning Library

CI-on-master passing | codecov 85% | docs passing | pypi package 1.1.0

The unified_planning library makes it easy to formulate planning problems and to invoke automated planners.

- Define problems in a *simple*, *intuitive*, and *planner independent* way
- Solve your planning problems using one of the native solvers, or by using any PDDL planner
- Dump your problems in PDDL (or ANML) format
- Parse PDDL problem formulations
- Simplification, grounding, removal of conditional effects and many other transformations are available
- and more...

### Usage

```
from unified_planning.shortcuts import *

x = Fluent("x")

a = InstantaneousAction("a")
a.add_precondition(Not(x))
a.add_effect(x, True)

problem = Problem("basic")
problem.add_fluent(x)
problem.add_action(a)
problem.set_initial_value(x, False)
problem.add_goal(x)

with OneshotPlanner(problem_kind=problem.kind) as planner:
    result = planner.solve(problem)
    if result.status in unified_planning.engines.results.POSITIVE_OUTCOMES:
        print(f"{planner.name} found this plan: {result.plan}")
    else:
        print("No plan found.")
```

https://github.com/aiplan4eu/unified-planning

# Competitions

**The International Planning Competition 2023**

- Classical Tracks
  - Daniel Fišer, Saarland University
  - Florian Pommerening, University of Basel
- Learning Tracks
  - Jendrik Seipp, Linköping University
  - Javier Segovia-Aguas, Universitat Pompeu Fabra
- Probabilistic Tracks
  - Ayal Taitler, University of Toronto
  - Scott Sanner, University of Toronto
- Numeric Tracks
  - Joan Espasa Arxer, University of St Andrews
  - Enrico Scala, University of Brescia
- HTN Tracks
  - Ron Alford, MITRE
  - Dominik Schreiber, Karlsruhe Institute of Technology
  - Gregor Behnke, University of Amsterdam

https://www.icaps-conference.org/competitions/

## International Planning Competition 2023 Classical Tracks

This is the website for the classical (sequential, deterministic) track of the IPC 2023. This is the 10th IPC containing classical tracks making it the oldest part of IPC.

### Results

The results were presented at the 33rd International Conference on Automated Planning and Scheduling on July 11 in Prague. The presentation slides of this talk contain additional details.

#### Optimal Track

- **Winner:** Ragnarok
  by Dominik Drexler, Daniel Gnad, Paul Höft, Jendrik Seipp, David Speck, and Simon Ståhlberg

#### Agile Track

- **Winner:** DecStar-2023
  by Daniel Gnad, Álvaro Torralba, and Alexander Shleyfman

#### Satisficing Track

- **Winner:** Scorpion Maidu and Levitron
  by Augusto B. Corrêa, Guillem Francès, Markus Hecher, Davide Mario Longo, and Jendrik Seipp

## Using IPC 2023 planners

All participating planners can be used as Apptainer images. To build such an image, install Apptainer and clone the corresponding planner repository. You can find links to them in the list of participants. Code repositories contain two branches: `ipc2023-classical` contains exactly the code version that ran in the IPC, while `latest` can contain additional bug fixes published after the competition. We recommend using `latest` for all experiments. Some code repositories contain multiple Apptainer files for different planners that share a code base. Use Apptainer to build the planner from one of those recipes like this:

```
git clone https://github.com/ipc2023-classical/planner8.git
cd planner8
sudo apptainer build maidu_sat.sif Apptainer.maidu_sat
```

Some planners require CPLEX to run. To use them, you have to acquire a CPLEX license (IBM offers free academic licenses) and download the installer file `cplex_studio2211.linux_x86_64.bin`. Place it into a directory and use the environment variable `IPC_THIRD_PARTY` to identify it.

```
export IPC_THIRD_PARTY=/path/to/some/directory
cp cplex_studio2211.linux_x86_64.bin $IPC_THIRD_PARTY

git clone https://github.com/ipc2023-classical/planner17.git
cd planner17
sudo apptainer build ragnarok.sif Apptainer.ragnarok
```

All planners are also available through planutils.

Source: https://ipc2023-classical.github.io

# Classical Planning Summary

- Goal-directed planning
  - STRIPS, PDDL
- Search-based approaches:
  - Forward (progression)
  - Backward (regression)
- SAT-based approaches:
  - Encode as SAT (CNF)
  - Solved by SATPlan and other SAT planners
- Other approaches
  - Planning graph – encodes constraints, derives heuristics
  - Situation calculus – FOL representation of actions
  - CSP formulation – bounded planning problems
  - Partial-order planning – plans as graphs; supports parallelism and explainability

# Characteristics of Planning

- Algorithm Properties
  - Soundness: All generated plans are valid solutions.
  - Completeness: If a solution exists, it will be found.
  - Optimality: Best plan found if admissible heuristics are used.
- Complexity
  - PlanSAT: Is there any solution?
  - Bounded PlanSAT: Is there a solution of length $\leq k$?
  - Both decidable (finite states); in general, in PSPACE.
  - Bounded PlanSAT often NP-complete; PlanSAT is in P.
  - Optimal planning is hard; sub-optimal planning often easier.
  - **In practice: effective heuristics and alternative planning models help**

# Modern “Classical” Planning

# LLM-Guided PDDL Creation and Refinement

- Goal
  - Automate PDDL domain/problem generation and refine models with feedback.
- Approach
  - LLM drafts initial PDDL from natural language goals domain and task descriptions
  - Evaluate generated PDDL in environment.
  - Iterative refinement via environment testing.
  - Planner executes final PDDL models.
- Example
  - *Mahdavi et al.,* NeurIPS 2024 NeurIPS | GitHub
  - LLM + feedback loop solves 66% tasks vs GPT-4's 29% intrinsic planning with CoT
  - Environment feedback greatly boosts planning success over LLM-only approaches
  - Planner: Fast Downward (with TextWorld modifications); plans validated with VAL.

# Task Decomposition & Subgoal Planning

- Goal
  - Combine LLM commonsense reasoning with symbolic planners for complex tasks.
- Approach
  - LLM decomposes high-level goals into subgoals.
  - Subgoals encoded in PDDL and solved by planners.
  - Complex subgoals handled by Monte Carlo Tree Search (MCTS) with LLM policy.
- Example
  - *Kwon et al.*, ICRA 2025 ArXiv | Github
  - Hybrid symbolic + MCTS planning reduces time and boosts scalability (reduces planning time while maintaining competitive success rates vs. baseline).
  - Planner: PDDL-based symbolic planner (Fast Downward as baseline) + MCTS-based LLM planner for complex subgoals.

# Heuristic Generation via LLMs

- Goal
  - Leverage LLMs to create domain-specific heuristics for classical planners.
- Approach
  - LLM synthesizes Python-coded heuristics from domain specs.
  - Heuristics plugged into classical planners to guide search.
- Example
  - *Corrêa et al., 2025* ArXiv
  - LLM-generated heuristics reduce explored states significantly
  - Approach outperforms standard domain-independent heuristics.
  - Planner:  Pyperplan using Greedy Best-First Search (GBFS)

# Example: Generalized Planning in PDDL Domains with Pretrained Large Language Models (LLMs)

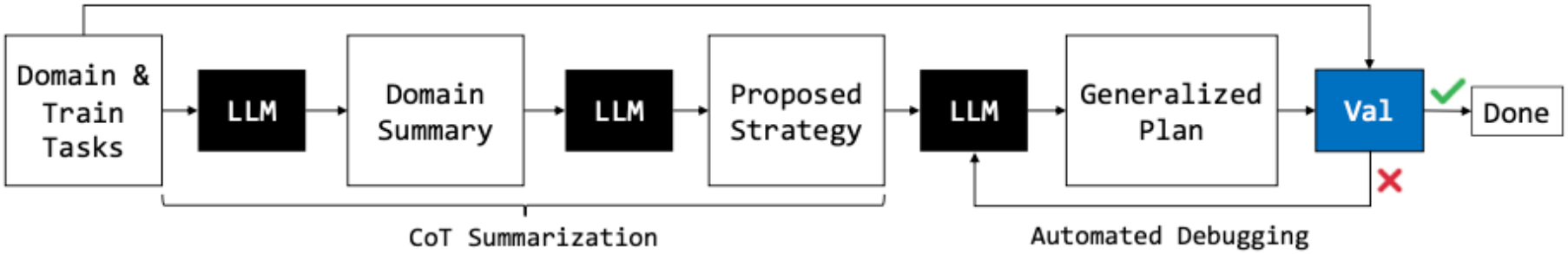



Figure 1: Overview of pipeline for generalized planning with pretrained LLMs. See text for details.

Source: Silver, T., S. Dan, K. Srinivas, J. B. Tenenbaum, L. Kaelbling and M. Katz (2024). Generalized planning in PDDL domains with pretrained large language models. Proceedings of the AAAI Conference on Artificial Intelligence. https://ojs.aaai.org/index.php/AAAI/article/view/30006

# Generalized Planning in PDDL Domains with LLMs

- Goal
  - Leverage LLMs as generalized planners to learn reusable strategies across PDDL domains.
- Approach
  - LLM (GPT-4) prompted with PDDL domain descriptions and two training tasks.
  - LLM + Chain-of-Thought extracts domain rules and strategies.
  - LLM generates domain-specific Python programs for new tasks.
  - Automated debugging refines outputs via re-prompting.
- Example
  - *Silver et al., AAAI 2024* AAAI
  - No classical planner used — LLM generated programs directly, compared against classical baselines.
  - LLM (GPT-4) generalized across 7 PDDL domains with only 2 training tasks; often rivaling or surpassing classical baselines when automated debugging was applied.

# From Classical to Modern Planning

- Classical planning remains highly relevant
  - Crucial in robotics, logistics, space mission planning, game AI, and multi-agent coordination, etc.
- Scaled via:
  - Heuristics from relaxed models and abstractions
  - Hierarchical decomposition (HTNs, HLAs)
  - Emerging LLM-assisted methods for PDDL model generation, subgoal planning, and heuristic learning
- Yet real world environments involve uncertainty
  - Noisy sensors, incomplete information, and stochastic dynamics
  - Goals often involve trade-offs among preferences, risk, and utility
- Next topics:
  - Representation, reasoning, decision making under uncertainty!

# Homework

- Readings:
  - Classical planning: RN: 11.1, 11.2
  - SATPlan: RN 7.7.4, 11.2.3
  - Complexity: RN Chapter 11, Bibliographical and Historical Notes (last page)
- Reviews:
  - Search (review): RN 3.3, 3.4, 3.5
  - Proposition logic and inference (review): RN: 7.3, 7.4, 7.5